PyTorch From Scratch

How Layers, Weights, and Training Actually Work — A Practical Guide

1. Overview

This guide summarizes the practical, implementation-level side of building neural networks in PyTorch: how `nn.Module` stores its layers and weights, how a forward/backward/optimizer step actually changes those weights in memory, how to reach into a model's submodules directly, and how to save and reload everything afterward. It's written around a concrete running example — an `Autoencoder` made of an `Encoder` and a `Decoder` — the same shape of model used throughout this tutorial track.

2. The Core Object: `nn.Parameter`

A `Tensor` is just a multi-dimensional array. `nn.Parameter` is a thin subclass of `Tensor` that adds exactly one thing: a flag marking it as trainable. When `nn.Linear(784, 400)` is constructed, internally it does roughly:

```
self.weight = nn.Parameter(torch.empty(400, 784))
self.bias = nn.Parameter(torch.empty(400))
```

That's the entire storage mechanism at its core — a `Parameter` is nothing more than a tensor of numbers plus that one flag.

3. The Mechanism Behind `self.` — `nn.Module.__setattr__`

`nn.Module` overrides Python's normal attribute-assignment behavior. Every time you write `self.something = X` inside a module:

- if `X` is an `nn.Parameter` → it's filed into an internal dict, `self._parameters`
- if `X` is itself an `nn.Module` (an `nn.Linear`, or a whole custom `Encoder`) → it's filed into `self._modules`
- otherwise → it's just a normal Python attribute, nothing special happens

This is *why* the `self.` convention matters in practice: `self.layer1 = nn.Linear(...)` triggers this registration, while a bare local variable `layer1 = nn.Linear(...)` never does — and unregistered weights are invisible to the optimizer, so they never train.

Because `self._modules` can hold other modules, and each of those has its own `_parameters` and `_modules`, this naturally builds a **tree**. An `Autoencoder` isn't a flat bag of numbers — it's a nested structure:

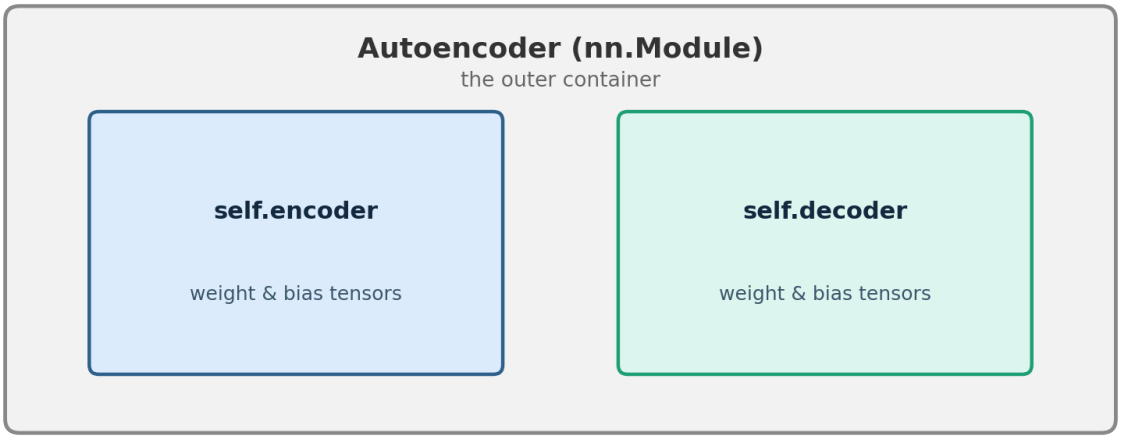


Figure 1: How an Autoencoder is structured internally: an outer container holding two independently-registered submodules, each with their own weight and bias tensors.

Seeing this for real

Run this on any model you've built:

```
for name, param in ae.named_parameters():
    print(name, param.shape)
```

which prints something like:

```
encoder.net.0.weight   torch.Size([400, 784])
encoder.net.0.bias     torch.Size([400])
encoder.net.2.weight   torch.Size([2, 400])
encoder.net.2.bias     torch.Size([2])
decoder.net.0.weight   torch.Size([400, 2])
decoder.net.0.bias     torch.Size([400])
decoder.net.2.weight   torch.Size([400, 784])
decoder.net.2.bias     torch.Size([784])
```

Each name is literally a path down the tree: `encoder.net.0.weight` means "go into `self.encoder`, then its `self.net` (the `Sequential`), then index `0` (the first `Linear`), then its `weight`." Indices `0` and `2` (not `1`) because index `1` is the `ReLU` — it has no parameters, so it's simply skipped by this listing.

`model.parameters()` — which you hand directly to the optimizer — is just this same recursive walk, minus the names. The optimizer stores references to those actual tensor objects (not copies), which is the only reason it's later able to modify them in place.

4. Composing Layers: `Linear`, `ReLU`, `Sequential`

- **`nn.Linear(in_features, out_features)`** computes $y = xW^T + b$: takes a vector of size `in_features`, returns one of size `out_features`, with a learnable weight matrix and bias vector in between.
- **`nn.ReLU()`** applies $\max(0, x)$ elementwise. This is not optional decoration: stacking two `Linear` layers with nothing nonlinear between them is mathematically identical to a *single* `Linear` layer, since a linear function of a linear function is still linear. The nonlinearity is what lets stacked layers represent anything beyond a single linear map.
- **`nn.Sequential(A, B, C)`** is a container that chains modules in the order given. `self.net(x)` where `net = nn.Sequential(A, B, C)` is exactly shorthand for:

```
h = A(x)
h = B(h)
h = C(h)
return h
```

  Nothing else is happening — it saves you from writing that chaining out by hand inside `forward`.

Tracing real shapes through an `Encoder`

Concretely, for one flattened MNIST image going through `Linear(784,400)` → `ReLU` → `Linear(400,2)`:

Every `nn.Linear(a, b)` you write only needs its two numbers to match its neighbors: `a` = whatever size arrived, `b` = whatever size you want to hand off next.

5. Output Activations Must Match the Loss

A plain `nn.Linear` with nothing after it can output *any* real number. For a decoder reconstructing pixel intensities scaled to $[0, 1]$ (via `transforms.ToTensor()`), an unconstrained output is a problem in two separate ways:

1. Range matching. The decoder's output needs to be comparable to the real pixel values it's scored against, which live in $[0, 1]$.

2. The loss function requires it. `F.binary_cross_entropy` computes `log(x_hat)` and `log(1 - x_hat)` for every pixel. `log` of a negative number or of zero is undefined — you'd get `nan`. So `x_hat` being strictly inside $(0, 1)$ isn't just tidy, it's a **hard mathematical requirement** for the loss to be computable at all, regardless of what the network has learned so far.

`nn.Sigmoid()`, applied as the very last layer, is what guarantees this:

$$\text{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

Note the **encoder's** output (the latent code `z`) is left unsquashed — there's no fixed target `z` needs to match, so nothing constrains its range. Sigmoid belongs at the end of the *decoder* only.

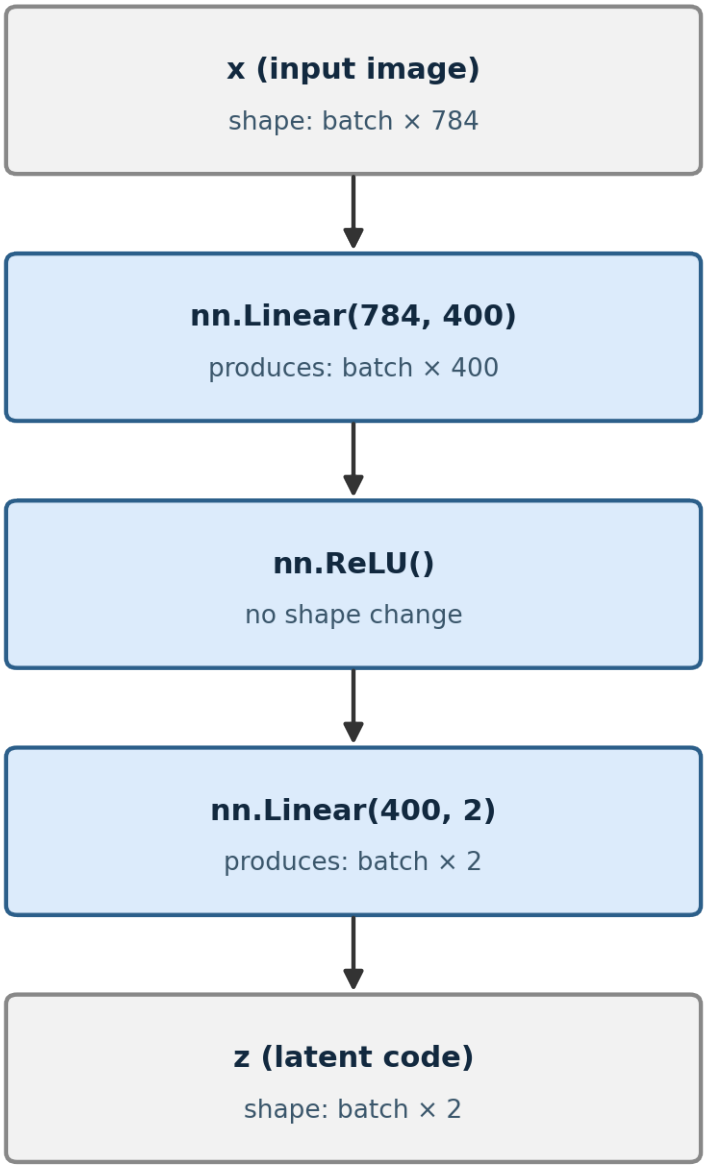


Figure 2: Every arrow shows a real tensor shape as it flows through the Encoder — note ReLU changes values but never the shape.

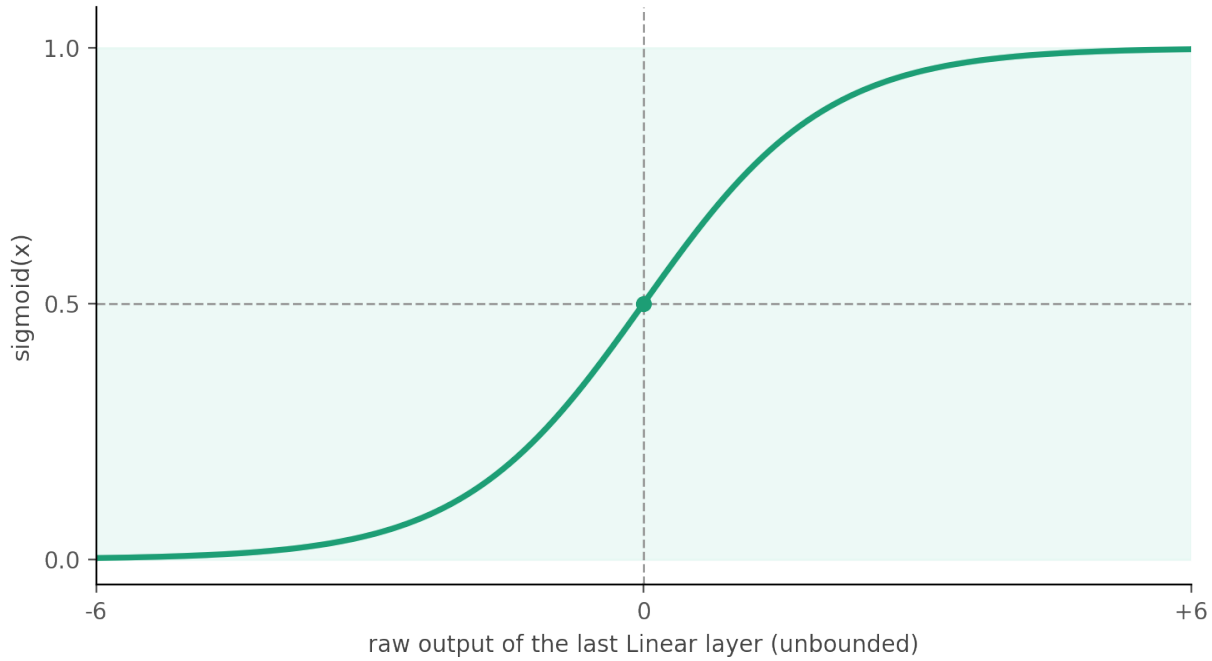


Figure 3: Sigmoid squashes any real number into (0, 1) — large negative inputs approach 0, large positive inputs approach 1, and 0 maps to exactly 0.5.

6. Reshaping Tensors: `.view()`

MNIST images arrive as `(batch, 1, 28, 28)` — one channel, 28×28 pixels — but a `Linear` layer expects a flat vector per example: `(batch, 784)`. `.view(...)` reinterprets a tensor's shape without changing or copying its underlying data (it requires the tensor to be memory-contiguous; if it isn't, `.reshape(...)` is the safer drop-in alternative, since it copies when necessary instead of erroring).

```
x_flat = x.view(x.size(0), -1)      # (batch, 1, 28, 28) -> (batch, 784)
```

`-1` tells PyTorch "infer this dimension automatically from the total number of elements." Using `x.size(0)` rather than hardcoding the batch size keeps this correct even for a final, smaller batch at the end of an epoch.

Going the other direction — turning a flat `(batch, 784)` reconstruction back into a displayable image — uses the same call with the shape reversed:

```
x_image = x_flat.view(-1, 1, 28, 28)    # (batch, 784) -> (batch, 1, 28, 28)
```

Here `-1` is inferred as the batch size instead — the same placeholder, just solving for whichever dimension you left out.

7. The Training Loop Lifecycle

A `Parameter` has two separate slots worth knowing about explicitly: `.data` (its current values) and `.grad` (a same-shaped tensor holding the gradient, or `None` before the first backward pass). The

standard four-line training step exists entirely because of this split:

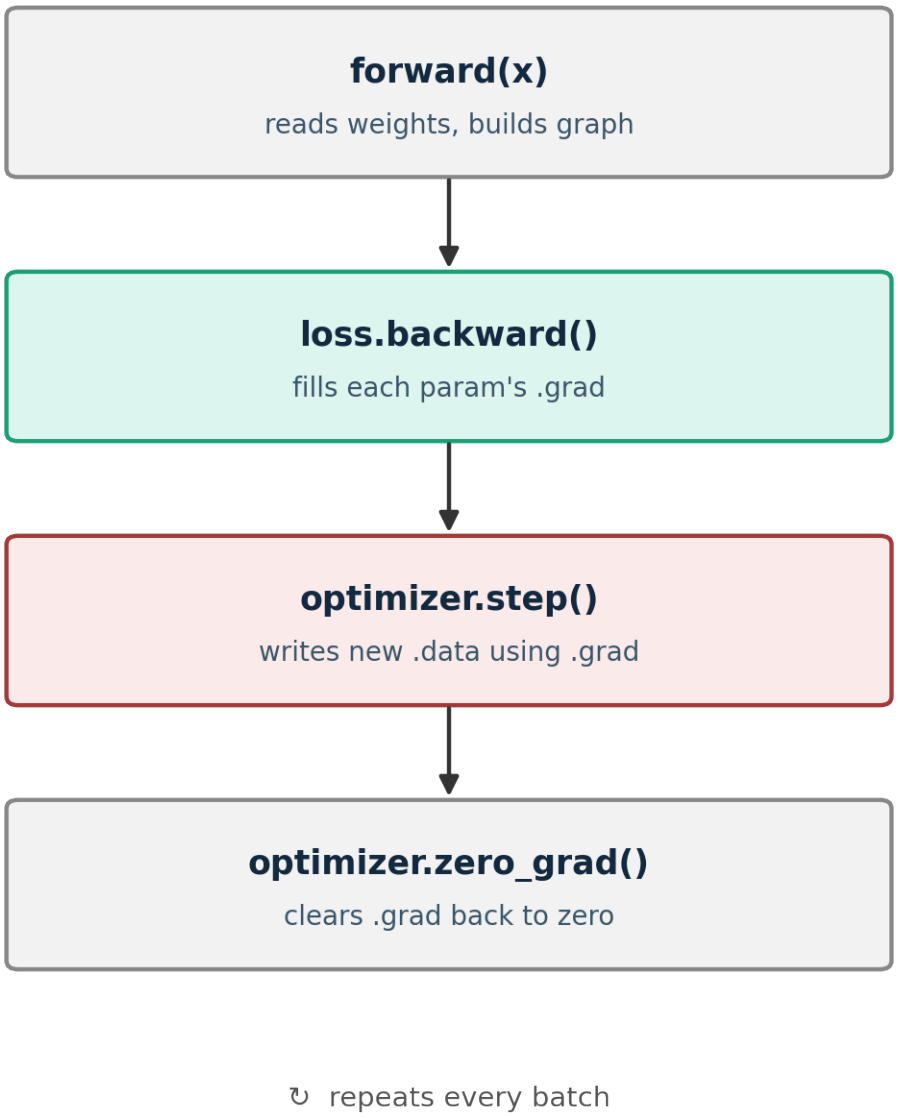


Figure 4: What each line of the training loop actually does to weights and gradients in memory.

- `forward(x)` reads `.data` from every layer to compute the output. Nothing about storage changes yet — but it silently builds a computation graph recording every operation performed, which is what makes the next step possible.
- `loss.backward()` walks that graph backward (this is backpropagation: the chain rule, applied systematically) and writes into `.grad` for every parameter involved. `.grad` **accumulates** (`+=`) rather than overwrites — a deliberate design choice that supports gradient accumulation across multiple mini-batches for those who want it — which is exactly why skipping `zero_grad()` silently corrupts training: old gradients keep adding onto new ones.

- `optimizer.step()` is the only line that touches `.data`. For plain SGD it's literally `p.data -= lr * p.grad` for every parameter `p` the optimizer was given; Adam does something more elaborate using running averages of `.grad`, but it is still, fundamentally, reading `.grad` and writing `.data`.
- `optimizer.zero_grad()` resets every `.grad` back to zero (or `None`), so the next batch's `backward()` starts clean.

8. Accessing Submodules Directly

Because `encoder` and `decoder` are registered as `self.encoder` / `self.decoder`, they are independently callable — you never have to go through the full `forward()` to use just one half of the model.

Get the latent code for real data (encoder only):

```
x_flat = x.view(x.size(0), -1).to(device)
z = ae.encoder(x_flat)
```

Decode a hand-picked latent code (decoder only, no encoder involved):

```
z_custom = torch.tensor([[1.5, -0.8]]).to(device)
x_hat = ae.decoder(z_custom)
```

VAE — the encoder returns two tensors instead of one:

```
mu, logvar = vae.encoder(x_flat)
z = reparameterize(mu, logvar)   # a random sample from the encoded distribution...
z_deterministic = mu             # ...or use mu directly, no sampling, for a stable point
x_hat = vae.decoder(z)
```

Using `mu` directly is what you want for a latent-space scatter plot — each point should sit at its distribution's *center*, not jump around randomly every time the cell re-runs.

VAE — generate brand-new digits (decoder only, no image involved):

```
z = torch.randn(8, LATENT_DIM).to(device)   # sample straight from the prior
samples = vae.decoder(z)
```

9. Collecting Results Across Batches

A `DataLoader` only ever hands you one batch at a time. Any time you need "one big result built from many small batches" — like every latent code for the entire test set — the pattern is always the same three steps:

```
results = []                        # 1. empty container, before the loop

for x, y in loader:
    value = compute_something(x) # one result per batch
    results.append(value)        # 2. accumulate, once per batch

results = torch.cat(results)     # 3. concatenate into one tensor, after the loop
```

Applied to collecting latent codes and labels for an entire test set:

```
all_z, all_labels = [], []

for x, y in test_loader:
    x_flat = x.view(x.size(0), -1).to(device)
    z = ae.encoder(x_flat)
    all_z.append(z.detach().cpu())
    all_labels.append(y)

latents = torch.cat(all_z).numpy()
labels = torch.cat(all_labels).numpy()
```

Why `.detach().cpu()` specifically: `z` comes out still attached to the autograd graph (PyTorch assumes you might want to backpropagate through it) and possibly living on the GPU. Since this is inference, not training, `.detach()` cuts it loose from the graph and `.cpu()` moves it back to regular memory — otherwise `torch.cat` and later plotting calls either fail outright or silently build up a large, unused computation graph.

10. Saving and Loading Models

Because a model is just a named tree of tensors, saving it is just serializing that tree.

Option A — `state_dict` (the standard approach; use this by default)

```
torch.save(ae.state_dict(), "ae_weights.pt")
```

`state_dict()` returns an ordered dict of exactly the `name -> tensor` pairs seen in Section 3 — no code, no architecture, just numbers. Loading requires a model with the *same* tree shape already constructed, ready to receive those numbers:

```
ae2 = Autoencoder().to(device)
ae2.load_state_dict(torch.load("ae_weights.pt", map_location=device))
ae2.eval()
```

If `ae2`'s architecture doesn't match the checkpoint — a different `hidden_dim`, a missing layer — `load_state_dict` raises an error, because some name in the saved dict has nowhere to go.

Option B — pickling the whole object (works, but more fragile)

```
torch.save(ae, "ae_full.pt")
ae2 = torch.load("ae_full.pt")   # no need to reconstruct the class first
```

This pickles the entire object, including a reference to the `Autoencoder` class itself. It's more convenient at load time, but it breaks if the class definition is later renamed, moved, or simply unavailable to import in whatever file you're loading from. **Prefer Option A** unless there's a specific reason not to.

Checkpointing for resuming training

If you want to pause and later resume training (rather than just deploy a finished model), bundle in the optimizer's state and the current epoch — Adam keeps its own running-average tensors internally, which would otherwise be lost on reload, causing training to effectively restart cold:

```
torch.save({
    "epoch": epoch,
    "model_state": ae.state_dict(),
    "optimizer_state": optimizer.state_dict(),
    "loss": epoch_loss,
}, "ae_checkpoint.pt")

# resuming later:
ckpt = torch.load("ae_checkpoint.pt", map_location=device)
ae.load_state_dict(ckpt["model_state"])
optimizer.load_state_dict(ckpt["optimizer_state"])
start_epoch = ckpt["epoch"] + 1
```

One easy detail to forget

Call `model.eval()` after loading if you're only doing inference (encoding or decoding, not training). Layers like batch norm and dropout behave differently in train vs. eval mode. A plain `Linear`/`ReLU` model is unaffected either way, but it's the right habit to build now, before adding convolutional layers or batch norm.

11. Quick Reference

Task	Code
Register a layer so it trains	`self.layer = nn.Linear(...)` (must use `self.`)
List every trainable parameter	`model.named_parameters()`
Flatten an image batch	`x.view(x.size(0), -1)`
Un-flatten back to images	`x.view(-1, 1, 28, 28)`
One training step	`loss.backward()` → `optimizer.step()` → `optimizer.zero_grad()`
Run only the encoder	`z = model.encoder(x)`
Run only the decoder	`x_hat = model.decoder(z)`
Collect results across all batches	empty list → `.append()` per batch → `torch.cat()` after
Save weights only (preferred)	`torch.save(model.state_dict(), path)`
Load weights only	`model.load_state_dict(torch.load(path))`
Save for resuming training	dict of `model_state` + `optimizer_state` + `epoch`
Switch to inference mode	`model.eval()`